

Kosh

Your money, explained back to you - with the working shown.

Simulates

hundreds of futures for every goal, every what-if

Recommends

only moves it has tested against your plan

Explains

every number traced, every assumption editable

THE PROBLEM

People can see their balance. They can't see their future.

- Banking and investing apps show **what you have**, not **whether it's enough**.
- "What if I lose my job?" or "can we afford the car?" needs a spreadsheet most people never build.
- Advice is either expensive, or free and quietly selling a product.
- Generic AI chatbots will happily invent a number.

"We earn well. I just don't know if we're doing okay."

- the sentence every one of our test households opened with

Three households, three different problems

Ananya, 27 · Pune

₹50k/month unplanned surplus, food delivery up 35%, a 42% credit card.

Retirement odds 4% → 69% with one step-up setting.

Farhan & Zoya, 36 · Bengaluru

Home loan, one child, 3 months of emergency money, ₹50 L cover vs ₹3.8 Cr needed.

College fund 36% → 72%, retirement 16% → 92%.

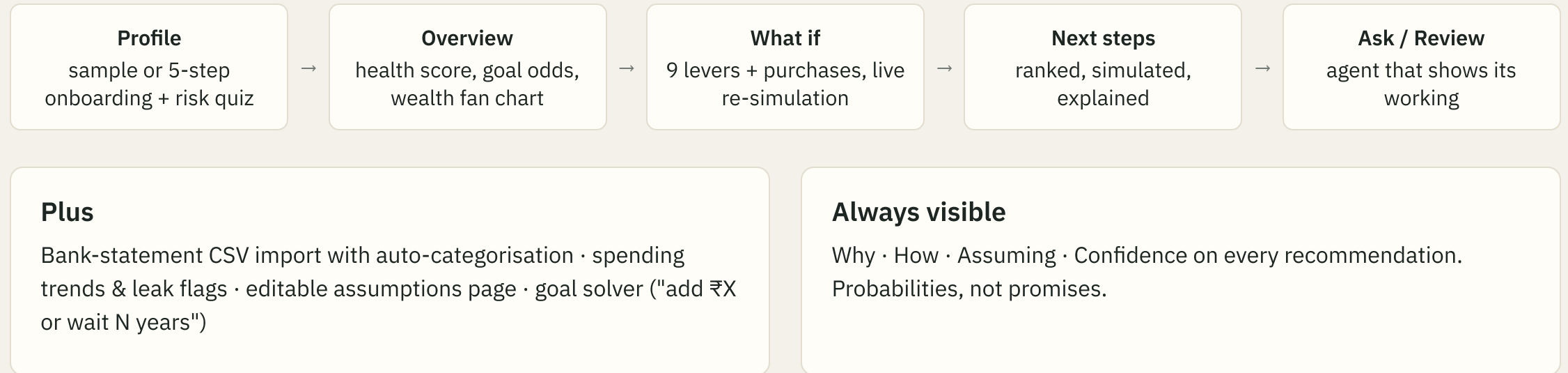
Mr. Iyer, 52 · Chennai

Saves 60% of income, nearly all in FDs and EPF. Six years to retirement.

Retirement 38% → 98% by automating his idle surplus.

All synthetic. Numbers from the live engine with default assumptions.

Five minutes from "am I okay?" to a plan



Every suggestion is simulated before it's shown

1. 11 generators spot situations: costly debt, thin cushion, idle cash, unplanned surplus, protection gaps, FD-heavy long-term money...
2. Each candidate is applied to a **copy** of the plan.
3. The whole plan is re-run - goals, health, cash flow.
4. Ranked by priority-weighted change in goal odds + safety + urgency.

Sheikh household · #1 "10% yearly step-up"

Retire at 60



Aarav's college fund



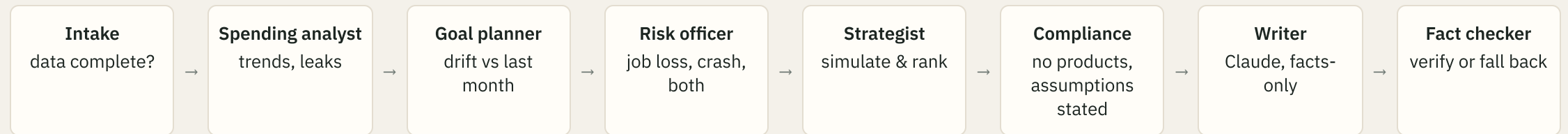
Same simulated markets before and after (common random numbers) - the difference is the change, not noise.

Claude plans and explains. The engine does the maths.

- **7 tools:** snapshot, run_scenario, check_goal, next_best_actions, analyze_spending, search_guides, get_assumptions.
- Multi-step tool loop; every step shown to the user.
- **Grounding check:** every ₹ and % in the answer must match a tool result. Untraced → sent back to the model once to fix.
- Refusal or outage → offline planner answers with the same tools.

```
◇ Understood: can I afford a purchase
→ get_snapshot   Health 56 · surplus ₹44,667/mo
→ run_scenario {big_purchase: ₹10 L, 1 yr}
    College fund 39% → 2%
→ search_guides "car loan emi"
✎ "You can pay for it, but it comes out of the college fund..."
✓ all 9 figures traced to calculations
```

The monthly review: eight specialists, one note



Proactive, not reactive: the review finds the problem before the user asks. Deterministic where it must be right, generative only where it must be human.

Assumptions are a page, not a footnote

Visible

Inflation 6%, education 9%, equity 11.5% $\pm$ 18%, surplus leakage 50%... each with a one-line reason.

Editable

Change any value and every probability, chart and recommendation re-runs.

Honest odds

We plan for ~80% odds, not the average (which is a coin flip). Goals keep their real asset mix - FDs stay FDs.

Categories, never products. Not a SEBI-registered adviser, and says so once - not on every line.

One container, or serverless on AWS

LAYER	SERVICE
Web	React on S3 + CloudFront (OAC)
API	Lambda (Node 22, arm64) via Function URL
AI	Claude on Amazon Bedrock
State	DynamoDB, on-demand, 30-day TTL
Ops	CloudWatch, X-Ray, error alarm
IaC	AWS SAM
CI/CD	GitHub Actions → OIDC → sam deploy → s3 sync → smoke test

Why a Function URL, not API Gateway?

Agent turns with several tool calls can exceed API Gateway's 30 s limit.

Why BM25, not a vector DB?

15 hand-written notes. Right tool for the size; Knowledge Bases is a one-function swap.

Same code everywhere

Laptop, Docker, this demo and Lambda run the identical Express app. The live demo is one container; the table is the production target, linted in CI.

Built so the numbers can be trusted

27

tests: engine properties,
agent, grounding, HTTP API

<0.6s

full dashboard: goals +
projection + 9 simulated
actions

0

numbers from the model that
aren't traced to a calculation

100%

demo works offline - no keys,
no Wi-Fi dependency

- Property tests: investing more never lowers odds; a crash never raises them; solver's answer actually reaches the target.
- Seeded Monte Carlo with common random numbers → deterministic, comparable, testable.

Where this goes

- **Banks & brokers:** an explainable planning layer inside existing apps - more SIP starts, step-ups and insurance attach, without product-pushing.
- **Employers:** financial wellness benefit; payroll data makes onboarding one click.
- **Advisers:** a first-pass plan and stress test before the meeting.

Next

- Account Aggregator (RBI AA framework) instead of CSV upload
- Tax engine: old vs new regime on real salary structure
- Historical bootstrap returns (fatter tails)
- Cognito auth, household sharing, monthly review by email

THANK YOU

**Kosh doesn't tell people what to buy.
It shows them what their choices do.**

kosh-2w6y.onrender.com

Repo · architecture doc · deployment guide - all in the submission.